

Dynamic Memory Allocation

Problem with Arrays

- Sometimes
 - Amount of data cannot be predicted beforehand
 - Number of data items keeps changing during program execution

Wastage of memory due to allocating more space beforehand.

Better Solution

- Dynamic memory allocation
 - Know how much memory is needed after the program is run
 - Example: ask the user to enter from keyboard
 - Dynamically allocate only the amount of memory needed
- C provides functions to dynamically allocate memory
 - `malloc`, `calloc`, `realloc`

Memory Allocation Functions

- `malloc`
 - Allocates requested number of bytes and returns a pointer to the first byte of the allocated space
- `calloc`
 - Allocates space for an array of elements, initializes them to zero and then returns a pointer to the memory.
- `free`
 - Frees previously allocated space.
- `realloc`
 - Modifies the size of previously allocated space.

Allocating a Block of Memory

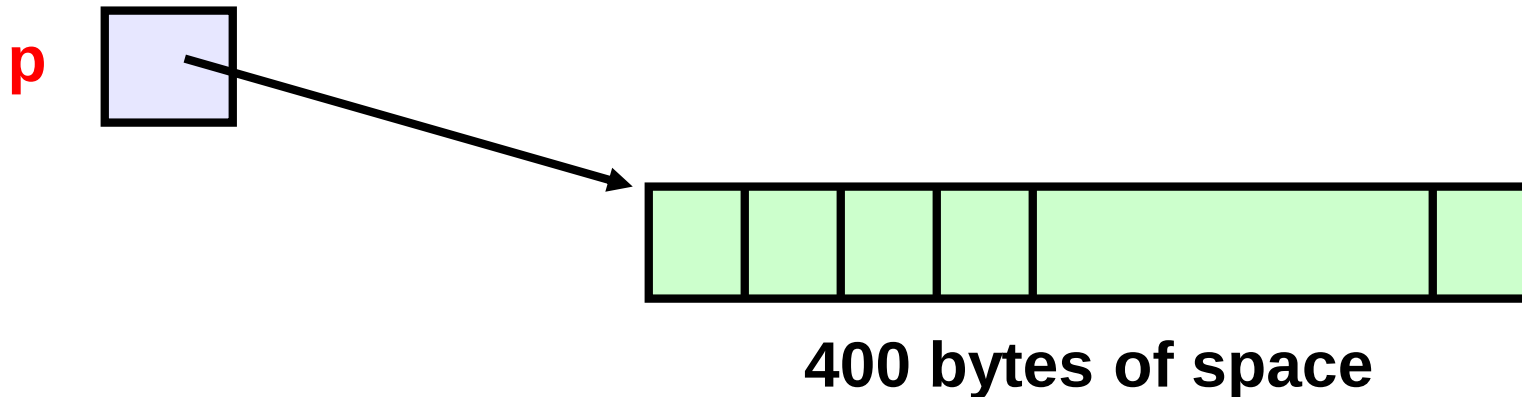
- A block of memory can be allocated using the function `malloc`
 - Reserves a block of memory of specified size and returns a pointer of type `void`
 - The return pointer can be type-casted to any pointer type
- General format:

```
type *p;  
p = (type *) malloc (byte_size);
```

Example

```
p = (int *) malloc(100 * sizeof(int));
```

- A memory space equivalent to **100 times the size of an int** bytes is reserved
- The address of the first byte of the allocated memory is assigned to the pointer **p** of type **int**



Contd.

- `cptr = (char *) malloc (20);`

Allocates 20 bytes of space for the pointer `cptr` of type `char`

- `sptr = (struct stud *) malloc(10*sizeof(struct stud));`

Allocates space for a structure array of 10 elements. `sptr` points to a structure element of type `struct stud`

Always use sizeof operator to find number of bytes for a data type, as it can vary from machine to machine

Points to Note

- `malloc` always allocates a block of contiguous bytes
 - The allocation can fail if sufficient contiguous memory space is not available
 - If it fails, `malloc` returns `NULL`

```
if ((p = (int *) malloc(100 * sizeof(int))) == NULL)
{
    printf ("\n Memory cannot be allocated");
    exit();
}
```

Dynamic allocation of memory for array

- Once the memory is allocated, it can be used with pointers, or with array notation
- Example:

```
int *p, n, i;  
scanf("%d", &n);  
p = (int *) malloc (n * sizeof(int));  
for (i=0; i<n; ++i)  
    scanf("%d", &p[i]);
```

The n integers allocated can be accessed as `*p, *(p+1), *(p+2),..., *(p+n-1)` or just as `p[0], p[1], p[2], ...,p[n-1]`

Example

```
int main()
{
    int i,N;
    float *height;
    float sum=0,avg;

    printf("Input no. of students\n");
    scanf("%d", &N);

    height = (float *)
        malloc(N * sizeof(float));
```

```
    printf("Input heights for %d
students \n",N);
    for (i=0; i<N; i++)
        scanf ("%f", &height[i]);

    for(i=0;i<N;i++)
        sum += height[i];

    avg = sum / (float) N;

    printf("Average height = %f \n",
        avg);

    free (height);
    return 0;
}
```

Releasing the Allocated Space: `free`

- An allocated block can be returned to the system for future use by using the `free` function
- General syntax:
 `free (ptr);`
 where `ptr` is a pointer to a memory block which has been previously created using `malloc`
- Note that no size needs to be mentioned for the allocated block, the system remembers it for each pointer returned

Can we allocate only arrays?

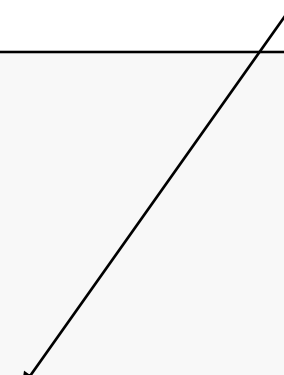
- malloc can be used to allocate memory for single variables also
 - `p = (int *) malloc (sizeof(int));`
 - Allocates space for a single int, which can be accessed as `*p`
- Single variable allocations are just special case of array allocations
 - Array with only one element

malloc()-ing array of structures

```
typedef struct{
    char name[20];
    int roll;
    float SGPA[8], CGPA;
} person;
void main()
{
    person *student;
    int i,j,n;
    scanf("%d", &n);
    student = (person *)malloc(n*sizeof(person));
    for (i=0; i<n; i++) {
        scanf("%s", student[i].name);
        scanf("%d", &student[i].roll);
        for(j=0;j<8;j++) scanf("%f", &student[i].SGPA[j]);
        scanf("%f", &student[i].CGPA);
    }
}
```

Static array of pointers

```
#define N 20
#define M 10
int main()
{
    char word[N], *w[M];
    int i, n;
    scanf("%d",&n);
    for (i=0; i<n; ++i) {
        scanf("%s", word);
        w[i] = (char *) malloc ((strlen(word)+1)*sizeof(char));
        strcpy (w[i], word) ;
    }
    for (i=0; i<n; i++) printf("w[%d] = %s \n",i,w[i]);
    return 0;
}
```



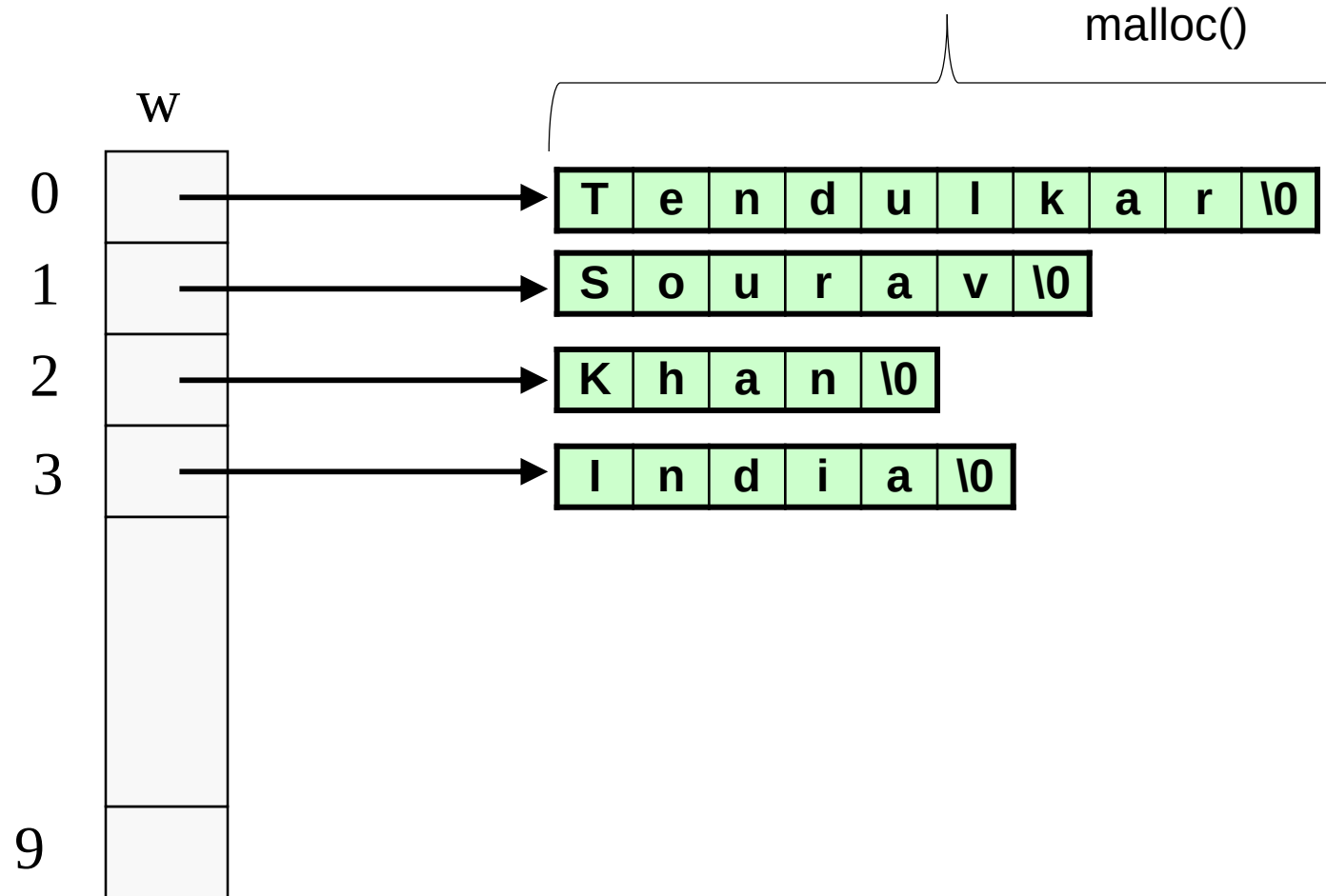
Static array of pointers

```
#define N 20
#define M 10
int main()
{
    char word[N], *w[M];
    int i, n;
    scanf("%d",&n);
    for (i=0; i<n; ++i) {
        scanf("%s", word);
        w[i] = (char *) malloc ((strlen(word)+1)*sizeof(char));
        strcpy (w[i], word) ;
    }
    for (i=0; i<n; i++) printf("w[%d] = %s \n",i,w[i]);
    return 0;
}
```

Output

```
4
Tendulkar
Sourav
Khan
India
w[0] = Tendulkar
w[1] = Sourav
w[2] = Khan
w[3] = India
```

How it will look like



Calloc() function

- Calloc function is used to allocate multiple blocks of contiguous memory in bytes. All the blocks are of same size.
- Syntax:

```
p = (type *) calloc(no. of blocks, int size);
```

Example:

```
p=(int *)calloc(5,10);
```

allocates 5 memory blocks of size 10 bytes.

Example – calloc()

```
#include<stdio.h>
#include<stdlib.h>
int main()
{
    int *p,i;
    p=(int *)calloc(10,sizeof(int));
    for(i=0;i<10;++i)
    {
        scanf("%d", &p[i]);
    }
    for(i=0;i<10;i++)
    {
        printf("%d ", p[i]);
    }
    return 0;
}
```

Realloc() function

- It is used to increase or decrease the size of memory already allocated by using malloc() or calloc()

Syntax: ptr=(type*)realloc(old_ptr, int newsize);

Example: p=(int*)malloc(10);
 q=(int*)realloc(p,30);

References

- https://cse.iitkgp.ac.in/~bivasm/pds_notes/dynamic_v1.ppt
- <https://www.learnpick.in/prime/documents/ppts/details/141/dynamic-memory-allocation-in-c>